
Batch Normalization using an SVD Layer

Abdulmohsen Mohammedsaleh, Majed Alsulami, Yousef Alogiley,
Jad Mounayer

abdulmohsen1284@gmail.com, majedsaadalsulami@gmail.com, yousef.alogiley@gmail.com,
Jad.mounayer@outlook.com

Abstract

In this first project week we studied the Rank Reduction Autoencoders paper and the reference implementation of the RR layer, reviewed Batch Normalization (BN), and ran a controlled experiment comparing a plain CNN (OA), a BN variant (OA_BN), and an RR variant (OA_RR) — all with identical initial weights — on a synthetic Gaussian-blob localization task. On this noiseless toy task the plain baseline is most accurate (mean test ℓ_2 error 2.6×10^{-3}), while the RR bottleneck (1.3×10^{-2}) clearly outperforms BN (3.4×10^{-2}).

📅 **Date:** July 16, 2026

Contents

1 Introduction 2

1.1 Work completed this week 2

2 Related Work 2

3 Methodology 2

3.1 Architecture code 3

4 Results 5

5 Discussion 5

6 Limitations and Future Work 5

7 Conclusion 5

References 6

1 Introduction

Batch Normalization (BN) [1] standardizes each feature of a mini-batch independently, ignoring correlations between features. The Rank Reduction (RR) layer, introduced with Rank Reduction Autoencoders (RRAEs) [2], offers a structured alternative: it computes a truncated SVD of the batch of activations and reconstructs them from the top- k singular directions — by the Eckart–Young–Mirsky theorem [3], the optimal rank- k approximation. The goal of this project is to study the RR layer as an SVD-based alternative to BN.

1.1 Work completed this week

1. **Paper study.** We read the RRAE paper [2] and worked through the underlying linear algebra of truncated SVD (Section 2).
2. **Code study.** We read the reference RR-layer implementation,¹ including its numerically stable SVD backward pass and how it builds a fixed *inference basis* by taking the SVD of the last 20 per-batch training bases.
3. **Batch Normalization.** We reviewed BN theory and implemented a BN variant of our test network.
4. **Experiment.** We compared BN against the RR layer in an otherwise-identical network (Section 3).

2 Related Work

Batch Normalization [1] normalizes each feature j of a mini-batch using its batch mean μ_j and variance σ_j^2 ,

$$\hat{x}_j = \frac{x_j - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}, \quad y_j = \gamma_j \hat{x}_j + \beta_j,$$

with learnable scale γ_j and shift β_j ; at inference, batch statistics are replaced by running averages. It operates per feature and is blind to correlations between features.

Truncated SVD. For a batch matrix $X \in \mathbb{R}^{M \times N}$ (one sample per column) with SVD $X = U \Sigma V^T$, keeping the top k components gives $\hat{X} = U_k \alpha$ with $\alpha = \Sigma_k V_k^T \in \mathbb{R}^{k \times N}$: each column of α is a k -dimensional embedding of one sample in the shared orthonormal basis U_k . By the Eckart–Young–Mirsky theorem [3], $\hat{X}$ is the best rank- k approximation of X .

Rank Reduction Autoencoders [2] embed this compress-and-reconstruct cycle inside a network as a differentiable layer: the RR layer outputs the full-size reconstruction $\hat{X}$ (not α), so it acts as a data-driven low-rank filter that preserves the shape and meaning of the activations it receives.

3 Methodology

Task. Regress the center (x_0, y_0) of a Gaussian blob from a 64×64 image; centers drawn uniformly from $[-0.5, 0.5]^2$; 800 training / 200 test samples, noiseless.

Models. All three share the same CNN feature extractor (three conv–ReLU–maxpool blocks, flattened to $M = 2048$ features) and regression head (Linear $2048 \rightarrow 128$, ReLU, Linear $128 \rightarrow 2$), and start from bit-identical weights (verified programmatically). They differ only in the bottleneck between extractor and head: none (**OA**), BatchNorm1d (**OA_BN**), or an RR layer with rank $k = 8$ (**OA_RR**).

¹https://github.com/JadM133/RR_layer

Training. 20 epochs, Adam [4] ($\text{lr } 10^{-3}$), MSE loss, batch size 32 on identical batches; PyTorch [5]. At evaluation, OA_RR finalizes its inference basis and OA_BN switches to running statistics.

3.1 Architecture code

All models are built from the same shared feature extractor:

```
1 def make_feature_extractor():
2     return nn.Sequential(
3         nn.Conv2d(1, 8, kernel_size=3, padding=1), nn.ReLU(inplace=True), nn.
4             MaxPool2d(2, 2),
5         nn.Conv2d(8, 16, kernel_size=3, padding=1), nn.ReLU(inplace=True), nn.
6             MaxPool2d(2, 2),
7         nn.Conv2d(16, 32, kernel_size=3, padding=1), nn.ReLU(inplace=True), nn.
8             MaxPool2d(2, 2),
9     )
```

The three variants differ only in the bottleneck inserted between the flattened features and the head:

```

7  class OA(nn.Module):                                # plain baseline
8      def __init__(self):
9          super().__init__()
10         self.features = make_feature_extractor()
11         self.classifier = nn.Sequential(
12             nn.Linear(32 * 8 * 8, 128), nn.ReLU(inplace=True),
13             nn.Linear(128, 2),
14         )
15
16     def forward(self, x):
17         x = self.features(x)
18         x = torch.flatten(x, 1)
19         return self.classifier(x)
20
21
22 class OA_BN(nn.Module):                              # BatchNorm bottleneck
23     def __init__(self):
24         super().__init__()
25         self.features = make_feature_extractor()
26         self.bottleneck = nn.BatchNorm1d(32 * 8 * 8)
27         self.classifier = ...                          # same head as OA
28
29     def forward(self, x):
30         x = self.features(x)
31         x = torch.flatten(x, 1)
32         x = self.bottleneck(x)
33         return self.classifier(x)
34
35
36 class OA_RR(nn.Module):                              # Rank Reduction bottleneck
37     def __init__(self, rank: int = 8):
38         super().__init__()
39         self.features = make_feature_extractor()
40         self.bottleneck = RRLayer(rank=rank)
41         self.classifier = ...                          # same head as OA
42
43     def forward(self, x):
44         x = self.features(x)
45         x = torch.flatten(x, 1)
46         x = self.bottleneck(x)
47         return self.classifier(x)

```

The core of the RR layer's training-mode forward pass (from the reference implementation) is:

```

48 X = torch.movedim(x, 0, -1).reshape(-1, batch_size)  # (M, N)
49 U, S, Vh = stable_SVD(X)
50 r = min(self.rank, S.shape[0])
51 basis = U[:, :r]                                       # top-r directions
52 coeffs = S[:r].unsqueeze(1) * Vh[:r]                 # alpha = Sigma_r V_r^T
53 X_hat = basis @ coeffs                                # rank-r reconstruction
54 self._basis_bank.append(basis.detach().cpu())         # for inference basis

```

4 Results

Table 1 Final metrics after 20 epochs (test set: 200 samples). Lower is better; best in **bold**.

Model	Final train MSE	Test MSE	Mean ℓ_2	Median ℓ_2
OA	2.95×10^{-6}	4.89×10^{-6}	0.00263	0.00235
OA_BN	1.33×10^{-3}	1.20×10^{-3}	0.03350	0.02616
OA_RR	1.07×10^{-4}	1.38×10^{-4}	0.01334	0.01140

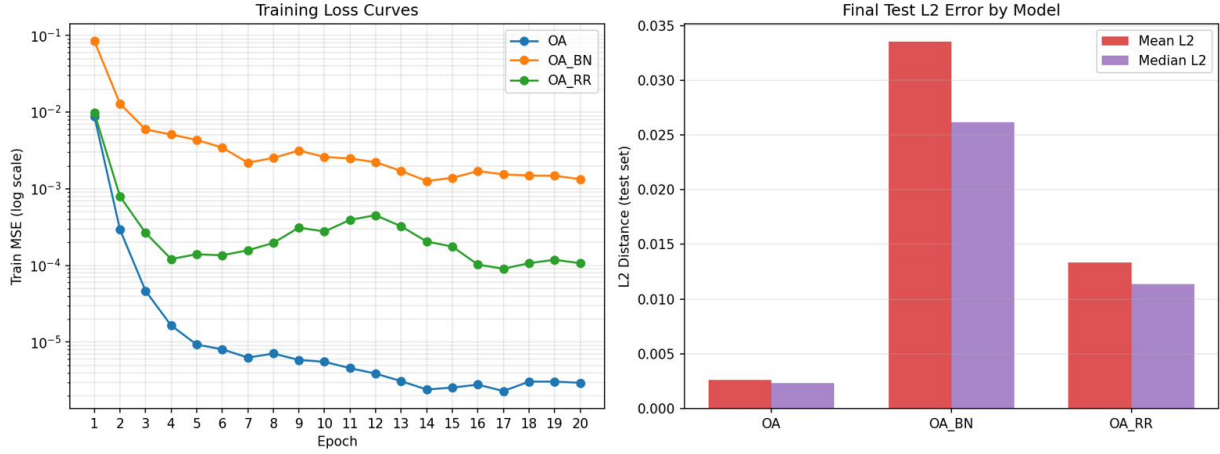


Figure 1 Training MSE per epoch (left, log scale) and final test ℓ_2 error (right).

5 Discussion

The plain baseline wins on this noiseless toy task, as expected: with nothing to denoise or regularize, any bottleneck only costs accuracy. The informative comparison is between the bottlenecks: the rank-8 RR layer degrades the mean error by $\sim 5\times$ relative to OA, versus $\sim 13\times$ for BN. We attribute this to the RR layer being a pure projection — it leaves the retained subspace untouched and its finalized inference basis stays close to the per-batch training bases — whereas BN rescales all 2048 features independently and swaps batch statistics for running averages at test time.

6 Limitations and Future Work

These results come from a single seed on a deliberately easy, noiseless task with one rank value ($k = 8$) and one bottleneck position. Next steps: repeat over multiple seeds; add input noise and/or reduce training data, where the RR layer’s denoising and regularization benefits should appear; sweep the rank k ; compare inference-basis strategies; and move toward the autoencoder setting of the original paper.

7 Conclusion

This week we studied the RRAE paper and the reference RR-layer code, reviewed and implemented Batch Normalization, and ran a controlled OA vs. OA_BN vs. OA_RR comparison from identical initial weights. On a noiseless localization task, the SVD-based RR bottleneck preserves

task-relevant information substantially better than BN ($\sim 2.5\times$ lower test error), motivating the noisier and harder settings we will study next.

References

- [1] Sergey Ioffe and Christian Szegedy. “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift”. In: *Proceedings of the 32nd International Conference on Machine Learning (ICML)*. 2015, pp. 448–456.
- [2] Jad Mounayer et al. *Rank Reduction Autoencoders*. 2024. arXiv: 2405.13980 [cs.LG]. URL: <https://arxiv.org/abs/2405.13980>.
- [3] Carl Eckart and Gale Young. “The approximation of one matrix by another of lower rank”. In: *Psychometrika* 1.3 (1936), pp. 211–218.
- [4] Diederik P. Kingma and Jimmy Ba. “Adam: A Method for Stochastic Optimization”. In: *International Conference on Learning Representations (ICLR)*. 2015.
- [5] Adam Paszke et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 32. 2019.